

Data 236- Distributed Systems

Homework-6

Kruthika Virupakshappa

Part 1

Database Design

```
from pymongo import MongoClient
from pymongo.errors import CollectionInvalid, OperationFailure
from .config import settings

_client = None

def get_client() -> MongoClient:
    global _client
    if _client is None:
        _client = MongoClient(settings.MONGODB_URI)
    return _client

def get_db():
    return get_client()[settings.MONGODB_DB]

def get_tasks_collection():
    return get_db()["tasks"]

def get_messages_collection():
    return get_db()["messages"]

def get_summaries_collection():
    return get_db()["summaries"]
```

```

def get_episodes_collection():
    return get_db()["episodes"]

def ensure_collections() -> None:
    db = get_db()

    tasks_schema = {
        "bsonType": "object",
        "required": ["title", "status", "priority", "dueDate", "category", "createdAt",
"updatedAt"],
        "properties": {
            "title": {"bsonType": "string", "maxLength": 100},
            "description": {"bsonType": ["string", "null"]},
            "status": {"enum": ["pending", "in-progress", "completed"]},
            "priority": {"enum": ["low", "medium", "high"]},
            "dueDate": {"bsonType": "date"},
            "category": {"enum": ["Work", "Personal", "Shopping", "Health", "Other"]},
            "createdAt": {"bsonType": "date"},
            "updatedAt": {"bsonType": "date"},
        },
    }

    if "tasks" not in db.list_collection_names():
        try:
            db.create_collection(
                "tasks",
                validator={"$jsonSchema": tasks_schema},
                validationLevel="strict",
            )
        except CollectionInvalid:
            pass
    else:
        try:
            db.command(
                {
                    "collMod": "tasks",
                    "validator": {"$jsonSchema": tasks_schema},
                    "validationLevel": "moderate",
                }
            )

```

```

    )
    except OperationFailure:
        # Non-admin users may not be allowed to modify validator; app-level
        # Pydantic validation still enforces constraints.
        pass

    get_tasks_collection().create_index("createdAt")
    get_tasks_collection().create_index("dueDate")

    get_messages_collection().create_index([("user_id", 1), ("session_id", 1),
("created_at", -1)])
    get_messages_collection().create_index([("user_id", 1), ("created_at", -1)])

    get_summaries_collection().create_index([("user_id", 1), ("scope", 1),
("session_id", 1), ("created_at", -1)])

    get_episodes_collection().create_index([("user_id", 1), ("created_at", -1)])
    get_episodes_collection().create_index([("user_id", 1), ("session_id", 1),
("created_at", -1)])

```

Backend Implementation

```

from fastapi import APIRouter, HTTPException, status
from pymongo.errors import PyMongoError

from ..db import get_tasks_collection
from ..schemas import TaskCreate, TaskOut, TaskUpdate
from ..utils import is_valid_object_id, now_utc, serialize_task, to_object_id

router = APIRouter(prefix="/api/tasks", tags=["Tasks"])

@router.post("", response_model=TaskOut, status_code=status.HTTP_201_CREATED)
def create_task(payload: TaskCreate):
    col = get_tasks_collection()
    doc = payload.model_dump()

    ts = now_utc()

```

```

    doc["createdAt"] = ts
    doc["updatedAt"] = ts

    try:
        result = col.insert_one(doc)
        created = col.find_one({"_id": result.inserted_id})
        return serialize_task(created)
    except pymongo.errors.PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while creating
task.")

@router.get("", response_model=list[TaskOut])
def get_all_tasks():
    col = get_tasks_collection()
    try:
        docs = list(col.find().sort("createdAt", -1))
        return [serialize_task(doc) for doc in docs]
    except pymongo.errors.PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while fetching
tasks.")

@router.get("/{task_id}", response_model=TaskOut)
def get_task(task_id: str):
    if not is_valid_object_id(task_id):
        raise HTTPException(status_code=400, detail="Invalid id format.")

    col = get_tasks_collection()
    try:
        doc = col.find_one({"_id": to_object_id(task_id)})
        if not doc:
            raise HTTPException(status_code=404, detail="Task not found.")
        return serialize_task(doc)
    except HTTPException:
        raise
    except pymongo.errors.PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while fetching
task.")

@router.put("/{task_id}", response_model=TaskOut)

```

```

def update_task(task_id: str, payload: TaskUpdate):
    if not is_valid_object_id(task_id):
        raise HTTPException(status_code=400, detail="Invalid id format.")

    col = get_tasks_collection()
    try:
        existing = col.find_one({"_id": to_object_id(task_id)})
        if not existing:
            raise HTTPException(status_code=404, detail="Task not found.")

        updates = payload.model_dump(exclude_unset=True)
        if not updates:
            raise HTTPException(status_code=400, detail="No fields provided to
update.")

        updates["updatedAt"] = now_utc()

        col.update_one({"_id": to_object_id(task_id)}, {"$set": updates})
        updated = col.find_one({"_id": to_object_id(task_id)})
        return serialize_task(updated)
    except HTTPException:
        raise
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while updating
task.")

@router.delete("/{task_id}", status_code=status.HTTP_204_NO_CONTENT)
def delete_task(task_id: str):
    if not is_valid_object_id(task_id):
        raise HTTPException(status_code=400, detail="Invalid id format.")

    col = get_tasks_collection()
    try:
        result = col.delete_one({"_id": to_object_id(task_id)})
        if result.deleted_count == 0:
            raise HTTPException(status_code=404, detail="Task not found.")
        return
    except HTTPException:
        raise
    except PyMongoError:

```

```
raise HTTPException(status_code=500, detail="Database error while deleting  
task.")
```

CRUD operations

1. POST /api/tasks

POST

/api/tasks

Create Task

Parameters

No parameters

Request body

required

application/json

Edit Value

Schema

{

"title": "Study Plan",

"description": "Create a study plan for exams",

"status": "pending",

"priority": "high",

"dueDate": "2026-03-17T16:55:28.868Z",

"category": "Work"

}

Execute

Clear

Responses

Curl

curl -X 'POST' \
 'http://127.0.0.1:8000/api/tasks' \
 -H 'accept: application/json' \
 -H 'Content-Type: application/json' \
 -d '{
 "title": "Study Plan",
 "description": "Create a study plan for exams",
 "status": "pending",
 "priority": "high",
 "dueDate": "2026-03-17T16:55:28.868Z",
 "category": "Work"
 }'

Request URL

http://127.0.0.1:8000/api/tasks

Server response

Code

Details

201

Response body

{
 "id": "69b987fe46510f4ac1d8d106",
 "title": "Study Plan",
 "description": "Create a study plan for exams",
 "status": "pending",
 "priority": "high",
 "dueDate": "2026-03-17T16:55:28.868000",
 "category": "Work",
 "createdAt": "2026-03-17T16:57:34.918000",
 "updatedAt": "2026-03-17T16:57:34.918000"
}

Response headers

access-control-allow-origin: *
content-length: 276
content-type: application/json
date: Tue, 17 Mar 2026 16:57:34 GMT
server: uvicorn

Responses

Code

Description

Links

201

Successful Response

No links

Media type

application/json

Controls

Accept header

Example Value

Schema

2. GET /api/tasks

GET/api/tasksGet All Tasks

Parameters

Cancel

No parameters

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/api/tasks' \
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/api/tasks
```

Server response

CodeDetails

200

Response body

```
[
  {
    "id": "69b98be5fecfa0a87f9d0e2",
    "title": "Get Groceries",
    "description": "Get weekly list of groceries",
    "status": "completed",
    "priority": "low",
    "dueDate": "2026-03-17T16:55:28.868000",
    "category": "Shopping",
    "createdAt": "2026-03-17T17:14:13.173000",
    "updatedAt": "2026-03-17T17:14:13.173000"
  },
  {
    "id": "69b987fe46510f4ac1d8d106",
    "title": "Study Plan",
    "description": "Create a study plan for exams",
    "status": "pending",
    "priority": "high",
    "dueDate": "2026-03-17T16:55:28.868000",
    "category": "Work",
    "createdAt": "2026-03-17T16:57:34.918000",
    "updatedAt": "2026-03-17T16:57:34.918000"
  }
]
```

Response headers

```
content-length: 562
content-type: application/json
date: Tue, 17 Mar 2026 17:15:05 GMT
server: uvicorn
```

Responses

CodeDescriptionLinks

200Successful ResponseNo links

Media type

3. GET /api/taks/{id}

GET

/api/tasks/{task_id}

Get Task

^

Parameters

Cancel

Name	Description
task_id required string (path)	69b98be5fecefa1a87f9d0e2

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/api/tasks/69b98be5fecefa1a87f9d0e2' \
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/api/tasks/69b98be5fecefa1a87f9d0e2
```

Server response

Code	Details
200	Response body <pre>{ "id": "69b98be5fecefa1a87f9d0e2", "title": "Get Groceries", "description": "Get weekly list of groceries", "status": "completed", "priority": "low", "dueDate": "2026-03-17T16:55:28.868000", "category": "Shopping", "createdAt": "2026-03-17T17:14:13.173000", "updatedAt": "2026-03-17T17:14:13.173000" }</pre> Download Response headers <pre>content-length: 283 content-type: application/json date: Tue, 17 Mar 2026 17:17:17 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value

Schema

```
{
  "id": "69b98be5fecefa1a87f9d0e2",
  "title": "Get Groceries",
  "description": "Get weekly list of groceries",
  "status": "completed",
  "priority": "low",
  "dueDate": "2026-03-17T16:55:28.868000",
  "category": "Shopping",
  "createdAt": "2026-03-17T17:14:13.173000",
  "updatedAt": "2026-03-17T17:14:13.173000"
}
```

4. PUT /api/tasks/{id}

PUT

/api/tasks/{task_id}

Update Task

⌵

Parameters

Cancel

Reset

Name	Description
task_id required	
string (path)	69b98be5fecefa1a87f9d0e2

Request body required

application/json

Edit Value | Schema

```
{  "title": "Groceries and clothes",  "description": "Get a few groceries and clothes ",  "status": "pending",  "priority": "medium",  "dueDate": "2026-03-17T17:22:20.776Z",  "category": "Shopping"}}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \  'http://127.0.0.1:8000/api/tasks/69b98be5fecefa1a87f9d0e2' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "title": "Groceries and clothes",  "description": "Get a few groceries and clothes ",  "status": "pending",  "priority": "medium",  "dueDate": "2026-03-17T17:22:20.776Z",  "category": "Shopping"}'
```

Request URL

http://127.0.0.1:8000/api/tasks/69b98be5fecefa1a87f9d0e2

Server response

Code	Details
200	Response body <pre>{ "id": "69b98be5fecefa1a87f9d0e2", "title": "Groceries and clothes", "description": "Get a few groceries and clothes ", "status": "pending", "priority": "medium", "dueDate": "2026-03-17T17:22:20.776000", "category": "Shopping", "createdAt": "2026-03-17T17:14:13.173000", "updatedAt": "2026-03-17T17:23:45.886000"}}</pre> Download Response headers <pre>access-control-allow-origin: * content-length: 296 content-type: application/json date: Tue, 17 Mar 2026 17:23:45 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

Example Value | Schema

```
{  "id": "string",  "title": "string",  "description": "string",  "status": "pending"}
```

5. DELETE /api/tasks/{id}

DELETE

/api/tasks/{task_id}

Delete Task

⌵

Parameters

Cancel

Name	Description
task_id * required	
string	69b98be5fecefa1a87f9d0e2
(path)	

Execute

Clear

Responses

Curl

curl -X 'DELETE' \
 'http://127.0.0.1:8000/api/tasks/69b98be5fecefa1a87f9d0e2' \
 -H 'accept: */*'

Request URL

http://127.0.0.1:8000/api/tasks/69b98be5fecefa1a87f9d0e2

Server response

Code	Details
204	Response headers access-control-allow-origin: * content-type: application/json date: Tue, 17 Mar 2026 17:27:27 GMT server: uvicorn

Responses

Code	Description	Links
204	Successful Response	No links
422	Validation Error	No links

Media type

application/json

Example Value | Schema

{
 "detail": [
 {
 "loc": [
 "string",
 0
],
 "msg": "string",
 "type": "string"
 }
]
}

6. Validation error

POST /api/tasks Create Task

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "title": "Get Groceries",
  "description": "random value without string",
  "status": "completed",
  "priority": "low",
  "dueDate": "2026-03-17T16:55:28.868Z",
  "category": "Shopping"
}
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/api/tasks' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "title": "Get Groceries",
    "description": "random value without string",
    "status": "completed",
    "priority": "low",
    "dueDate": "2026-03-17T16:55:28.868Z",
    "category": "Shopping"
  }'
```

Request URL

http://127.0.0.1:8000/api/tasks

Server response

Code	Details
422	Error: Unprocessable Content Response body <pre>{ "detail": "Validation error", "errors": [{ "field": "47", "message": "JSON decode error" }] }</pre> Download Response headers <pre>access-control-allow-origin: * content-length: 85 content-type: application/json date: Tue, 17 Mar 2026 17:32:02 GMT server: uvicorn</pre>

Responses

Code	Description	Links
201	Successful Response Media type application/json Controls Accept header. Example Value Schema <pre>{ "id": "string", "title": "string", "description": "string", "status": "pending", "priority": "low", "dueDate": "2026-03-17T17:32:02.191Z", "category": "Work", "createdAt": "2026-03-17T17:32:02.191Z", "updatedAt": "2026-03-17T17:32:02.191Z" }</pre>	

 No links || 422 | Validation Error | No links |

Sample Mongo DB data

```
❖ (env) Kruthika@Kruthikas-MacBook-Air Data 236 Distributed Sys % mongosh
Current Mongosh Log ID: 69b9918476a9d62637aa376d
Connecting to:      mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.8.1
Using MongoDB:      8.2.6
Using Mongosh:       2.8.1

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
  The server generated these startup warnings when booting
    2026-03-17T09:37:34.903-07:00: Access control is not enabled for the database. Read and write access to
    data and configuration is unrestricted
-----

test> use data236_hw6
switched to db data236_hw6
data236_hw6> db.tasks.find().pretty()
[
  {
    _id: ObjectId('69b987fe46510f4ac1d8d106'),
    title: 'Study Plan',
    description: 'Create a study plan for exams',
    status: 'pending',
    priority: 'high',
    dueDate: ISODate('2026-03-17T16:55:28.868Z'),
    category: 'Work',
    createdAt: ISODate('2026-03-17T16:57:34.918Z'),
    updatedAt: ISODate('2026-03-17T16:57:34.918Z')
  }
]
data236_hw6> 
```

Part 2: AI Memory FastAPI + MongoDB + Ollama local LLM)

1. /api/chat

POST /api/chat Chat

Parameters

No parameters

Request body required

application/json

Edit Value | Schema

```
{  "user_id": "Kruthika",  "session_id": "Planning phase",  "message": "I need help planning my study schedule for the upcoming exams"}  
```

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  'http://127.0.0.1:8080/api/chat' \  -H 'accept: application/json' \  -H 'Content-Type: application/json' \  -d '{  "user_id": "Kruthika",  "session_id": "Planning phase",  "message": "I need help planning my study schedule for the upcoming exams"}  '
```

Request URL

http://127.0.0.1:8080/api/chat

Server response

Code

Details

200

Response body

```
{  "reply": "To create a study schedule that works for you, I'll need to recall some previous information. It seems that I previously suggested creating a schedule with dedicated blocks for reviewing notes and practicing problems, setting specific and achievable goals for each study session, and prioritizing the most challenging subjects or topics before less demanding ones.\n\nHere's a sample schedule you can use as a starting point:\n\n**Block 1: Review and Note-taking (30 minutes - 1 hour)**\n\nReview notes from previous classes or study sessions\nSummarize key points in your own words\nOrganize notes into a concise and easily accessible format\n\n**Block 2: Practice Problems (30 minutes - 1 hour)**\n\nWork on practice problems or past exams\nFocus on understanding concepts and applying them to different scenarios\nTake breaks to avoid burnout and maintain focus\n\n**Block 3: Goal-Oriented Study (30 minutes - 1 hour)**\n\nSet specific and achievable goals for the study session\nFocus on the most challenging topics or subjects\nUse flashcards, concept maps, or other study aids to reinforce learning\n\n**Block 4: Break and Relaxation (15-30 minutes)**\n\nTake a break to stretch, move around, or relax\nAvoid screens and electronic devices during this time\nUse this opportunity to recharge and refocus\n\n**Block 5: Review and Reflection (30 minutes - 1 hour)**\n\nReview what you've learned during the study session\nReflect on what worked well and what didn't\nAdjust your study plan as needed to improve future sessions\n\nFeel free to adjust this schedule to fit your needs and preferences. Remember to prioritize your most challenging subjects or topics first, and take regular breaks to avoid burnout. Good luck with your exams!",  "memory_used": {    "short_term_count": 7,    "short_term_messages": [      {        "role": "user"      }    ]  }}
```

2. api/memory/{user_id}

GET

/api/memory/{user_id}

Memory View

⌵

Parameters

Cancel

Name	Description
user_id * required	
string (path)	Kruthika
session_id	
string (string null) (query)	session_id

Execute

Clear

Responses

Curl

```
curl -X 'GET' \  
'http://127.0.0.1:8000/api/memory/Kruthika' \  
-H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/api/memory/Kruthika
```

Server response

Code	Details
200	Response body <pre>{ "user_id": "Kruthika", "session_id": "default", "recent_messages": [], "latest_session_summary": null, "latest_lifetime_summary": { "user_id": "Kruthika", "session_id": null, "scope": "user", "text": "**Compact Long-Term User Profile Summary**\n\n**User Profile:**\n\n**Goals:** Plan a study schedule for upcoming exams and focus on data structures.\n\n**Preferences:**\n\n+ Moderate study sessions (2-4 hours/day)\n+ Quality over quantity\n+ Regular breaks to avoid burnout\n\n**Unresolved Tasks:**\n\n+ Create a personalized study schedule\n\n+ Identify specific areas of data structures to focus on\n\n+ Adjust study schedule as needed\n\n**Key Takeaways:**\n\n+ The user values quality over quantity in their study sessions.\n\n+ They prioritize regular breaks to maintain productivity and avoid burnout.\n\n+ The user is looking for a personalized study schedule and guidance on focusing on specific areas of data structures.", "created_at": "2026-03-17T18:17:09.693000" }, "recent_episodes": [] }</pre> Download Response headers <pre>content-length: 982 content-type: application/json date: Tue, 17 Mar 2026 18:19:50 GMT server: uvicorn</pre>

Responses

Code	Description	Links
200	Successful Response	No links

Media type

Controls Accept header.

3. api/aggregate/{user_id}

GET

/api/aggregate/{user_id} Aggregate View

⌵

Parameters

Cancel

Name	Description
user_id required	
string	
(path)	Kruthika

Execute

Clear

Responses

Curl

curl -X 'GET' \n'http://127.0.0.1:8000/api/aggregate/Kruthika' \n-H 'accept: application/json'

Request URL

http://127.0.0.1:8000/api/aggregate/Kruthika

Server response

Code	Details
200	Response body <pre>"user_id": "Kruthika", "daily_message_counts": [{ "date": "2026-03-17", "count": 14 }], "recent_summaries": [{ "user_id": "Kruthika", "session_id": null, "scope": "user", "text": "**Compact Long-Term User Profile Summary**\n\n**User Profile:**\n\n**Goals:** Plan a study schedule for upcoming exams and focus on data structures.\n\n**Preferences:**\n\n\tModerate study sessions (2-4 hours/day)\n\tQuality over quantity\n\tRegular breaks to avoid burnout\n\n**Unresolved Tasks:**\n\n\tCreate a personalized study schedule\n\tIdentify specific areas of data structures to focus on\n\tAdjust study schedule as needed\n\n**Key Takeaways:**\n\n\tThe user values quality over quantity in their study sessions.\n\tThey prioritize regular breaks to maintain productivity and avoid burnout.\n\tThe user is looking for a personalized study schedule and guidance on focusing on specific areas of data structures.", "created_at": "2026-03-17T18:17:09.693000" }, { "user_id": "Kruthika", "session_id": "Planning phase", "scope": "session", "text": "Here are the concise bullets summarizing the conversation:\n\n**User Goals:**\n\n\tPlan a study schedule for upcoming exams\n\tFocus on data structures for exams" }] </pre> Download

Response headers

```
content-length: 1751
content-type: application/json
date: Tue, 17 Mar 2026 18:22:42 GMT
server: uvicorn
```

Responses

Code	Description	Links
200	Successful Response	No links

Media type

application/json

Controls Accept header.

4. Messages

```
(env) Kruthika@Kruthikas-MacBook-Air Data 236 Distributed Sys % mongosh
data236_hm> db.messages.find({ user_id: 'Kruthika' }).pretty()
{
  "_id": ObjectId("60899aeca4b77c7a5de3b9"),
  "user_id": "Kruthika",
  "session_id": "Planning phase",
  "role": "assistant",
  "content": "Here are some last-minute preparation tips to help you feel more confident and prepared for your exams!\n" +
    "\n" +
    "1. **Review your notes**: Go through your notes one last time, and make sure you understand the key concepts and formulas.\n" +
    "2. **Practice, practice, practice**: Practice problems and past exams will help you identify areas where you need to focus your studying.\n" +
    "3. **Get enough sleep**: Make sure you get plenty of sleep before the exam to ensure you're well-rested and alert.\n" +
    "4. **Stay hydrated and energized**: Bring water and snacks to the exam to keep you hydrated and energized throughout the test.\n" +
    "5. **Arrive early**: Plan to arrive at the exam location early to get settled and composed before the test begins.\n" +
    "6. **Use the process of elimination**: If you're unsure of an answer, try to eliminate any obviously incorrect options and make an educated guess.\n" +
    "7. **Manage your time effectively**: Make sure you allocate enough time for each question, and don't spend too much time on any one question.\n" +
    "8. **Stay calm and focused**: Take deep breaths, and try to stay calm and focused throughout the test.\n" +
    "9. **Use the resources provided**: Make sure you use the resources provided, such as calculators or formulas, to help you solve problems.\n" +
    "10. **Believe in yourself**: You've prepared well, and you're ready to do your best. Believe in yourself, and trust that you'll perform well.\n" +
    "\n" +
    "Additionally, here are some specific tips for data structures exams:\n" +
    "\n" +
    "• **Focus on the most common data structures**: Make sure you understand the most common data structures, such as arrays, linked lists, stacks, and queues.\n" +
    "• **Practice implementing data structures**: Practice implementing data structures in code to reinforce your understanding.\n" +
    "• **Review the algorithms**: Review the algorithms for each data structure, including time and space complexity.\n" +
    "• **Focus on the applications**: Focus on the applications of each data structure, and how they're used in real-world scenarios.\n" +
    "\n" +
    "I hope these tips help you feel more confident and prepared for your exams!",
  "created_at": ISODate("2026-03-17T18:18:20.702Z")
},
  "_id": ObjectId("60899adb4b77c7a5de3bb"),
  "user_id": "Kruthika",
  "session_id": "Planning phase",
  "role": "user",
  "content": "How long should I study?",
  "created_at": ISODate("2026-03-17T18:18:43.311Z")
},
  "_id": ObjectId("60899a9dad4b77c7a5de3b4"),
  "user_id": "Kruthika",
  "session_id": "Planning phase",
  "role": "assistant",
  "content": "The ideal number of study hours per day can vary depending on individual factors, such as your learning style, schedule, and goals. However, here are some general guidelines:\n" +
    "\n" +
    "• **For a light study session**: 1-2 hours per day, focusing on reviewing notes, practicing problems, or working on a specific topic.\n" +
    "• **For a moderate study session**: 2-4 hours per day, covering a range of topics, practicing problems, and reviewing notes.\n" +
    "• **For an intense study session**: 4-6 hours per day, focusing on a specific topic or subject, practicing problems, and reviewing notes.\n" +
    "\n" +
    "It's essential to remember that:\n" +
    "\n" +
    "• **Quality over quantity**: It's better to study for shorter periods with focused attention than to study for long periods with distractions.\n" +
    "• **Consistency**: Aim to study at the same time every day to establish a routine and make progress.\n" +
    "• **Rest and recharge**: Take regular breaks to avoid burnout and maintain focus.\n" +
    "• **Sleep and self-care**: Ensure you get enough sleep (7-9 hours) and prioritize self-care activities, such as exercise, meditation, or spending time with loved ones.\n" +
    "\n" +
    "For data structures, I recommend starting with a moderate study session of 2-4 hours per day, focusing on reviewing notes, practicing problems, and working on specific topics. As you progress, you can adjust the duration and intensity of your study sessions based on your needs and progress.\n" +
    "\n" +
    "Remember, it's essential to find a balance between studying and other aspects of your life. Don't overstudy, as this can lead to burnout and decreased productivity.",
  "created_at": ISODate("2026-03-17T18:18:53.589Z")
},
  "_id": ObjectId("60899a8da4b77c7a5de3b3"),
  "user_id": "Kruthika",
  "session_id": "Planning phase",
  "role": "user",
  "content": "How many hours per day should I study?",
  "created_at": ISODate("2026-03-17T18:16:40.622Z")
},
  "_id": ObjectId("60899a8da4b77c7a5de3af"),
  "user_id": "Kruthika",
  "session_id": "Planning phase",
  "role": "assistant",
  "content": "For data structures, here are some key topics to focus on:\n" +
    "\n" +
    "1. **Arrays**: \n" +
    "   - Basic operations: indexing, slicing, concatenation\n" +
    "   - Advanced operations: dynamic memory allocation, bounds checking\n" +
    "   - Applications: searching, sorting, and manipulating arrays\n" +
    "2. **Linked Lists**: \n" +
    "   - Singly linked lists: insertion, deletion, traversal\n" +
    "   - Doubly linked lists: insertion, deletion, traversal, and efficient searching\n" +
    "   - Applications: dynamic memory allocation, stack and queue implementations\n" +
    "3. **Stacks and Queues**: \n" +
    "   - Basic operations: push, pop, peek, isEmpty\n" +
    "   - Advanced operations: implementing using linked lists or arrays\n" +
    "   - Applications: parsing, evaluating postfix expressions, and job scheduling\n" +
    "4. **Trees**: \n" +
    "   - Binary Trees: \n" +
    "     - Binary Search Trees (BSTs): insertion, deletion, traversal\n" +
    "     - AVL Tree and Red-Black Tree: balancing and self-balancing\n" +
    "     - Heaps: building, extracting the minimum/maximum element\n" +
    "   - Applications: file systems, database indexing, and priority queuing\n" +
    "5. **Graphs**: \n" +
    "   - Basic operations: adjacency matrix, adjacency list, traversal\n" +
    "   - Advanced operations: depth-first search (DFS), breadth-first search (BFS), and shortest paths\n" +
    "   - Applications: social networks, web pages, and network routing\n" +
    "6. **Hash Tables**: \n" +
    "   - Basic operations: insertion, deletion, lookup\n" +
    "   - Advanced operations: collision resolution, resizing, and caching\n" +
    "   - Applications: caching, data compression, and set operations\n" +
    "7. **Sorting and Searching**: \n" +
    "   - Basic sorting algorithms: bubble sort, selection sort, insertion sort\n" +
    "   - Advanced sorting algorithms: merge sort, quick sort, and heap sort\n" +
    "   - Searching algorithms: linear search, binary search, and interpolation search\n" +
    "\n" +
    "Focus on understanding the underlying data structures, their properties, and their applications. Practice implementing these data structures in code to reinforce your understanding.",
  "created_at": ISODate("2026-03-17T18:16:00.232Z")
},
  "_id": ObjectId("60899a52ad4b77c7a5de3ae"),
  "user_id": "Kruthika",
  "session_id": "Planning phase",
  "role": "user",
  "content": "What topics should I focus on for data structures?",
  "created_at": ISODate("2026-03-17T18:15:46.239Z")
},
  "_id": ObjectId("608999dad4b77c7a5de3ae"),
  "user_id": "Kruthika",
  "session_id": "Planning phase",
  "role": "assistant",
  "content": "To create a study schedule that works for you, I'll need to recall some previous information. It seems that I previously suggested creating a schedule with dedicated blocks for reviewing notes and practicing problems, setting specific and achievable goals for each study session, and prioritizing the most challenging subjects or topics before less demanding ones.\n" +
    "\n" +
    "Here's a sample schedule you can use as a starting point:\n" +
    "\n" +
    "• **Block 1: Review and Note-taking (30 minutes - 1 hour)**\n" +
    "  - Review notes from previous classes or study sessions\n" +
    "  - Summarize key points in your own words\n" +
    "  - Organize notes into a concise and easily accessible format\n" +
    "\n" +
    "• **Block 2: Practice Problems (1-2 hours)**\n" +
    "  - Focus on a specific topic or subject\n" +
    "  - Practice implementing data structures in code\n" +
    "  - Review and debug your solutions\n" +
    "\n" +
    "• **Block 3: Self-reflection and Planning (15-20 minutes)**\n" +
    "  - Reflect on your progress and challenges\n" +
    "  - Plan your next study session\n" +
    "  - Set specific goals for the next session\n" +
    "\n" +
    "Remember, consistency is key. Stick to your schedule as much as possible, and adjust it as needed based on your progress and needs. Good luck!
```

5. Summaries

```
data236_hw6> db.summaries.find({ user_id: "Kruthika"}).pretty()
[
  {
    _id: ObjectId('69b99aa1aadc871ae81324e'),
    session_id: 'Planning phase',
    scope: 'session',
    user_id: 'Kruthika',
    created_at: ISODate('2026-03-17T18:17:05.248Z'),
    text: 'Here are the concise bullets summarizing the conversation:\n' +
      '\n' +
      '**User Goals:**\n' +
      '\n' +
      '* Plan a study schedule for upcoming exams\n' +
      '* Focus on data structures for exams\n' +
      '\n' +
      '**User Preferences:**\n' +
      '\n' +
      '* Moderate study sessions (2-4 hours per day)\n' +
      '* Focus on reviewing notes, practicing problems, and working on specific topics\n' +
      '* Prioritize quality over quantity\n' +
      '* Take regular breaks to avoid burnout\n' +
      '\n' +
      '**Unresolved Tasks:**\n' +
      '\n' +
      '* Create a personalized study schedule based on user's specific needs and goals\n' +
      '* Identify specific areas of data structures to focus on (e.g., arrays, linked lists, trees, graphs, hash tables)\n' +
      '* Adjust study schedule as needed to improve progress and avoid burnout'
  },
  {
    _id: ObjectId('69b99aa5aadc871ae81324f'),
    session_id: null,
    scope: 'user',
    user_id: 'Kruthika',
    created_at: ISODate('2026-03-17T18:17:09.693Z'),
    text: '**Compact Long-Term User Profile Summary**\n' +
      '\n' +
      '**User Profile:**\n' +
      '\n' +
      '* **Goals:** Plan a study schedule for upcoming exams and focus on data structures.\n' +
      '* **Preferences:**\n' +
      '\t+ Moderate study sessions (2-4 hours/day)\n' +
      '\t+ Quality over quantity\n' +
      '\t+ Regular breaks to avoid burnout\n' +
      '* **Unresolved Tasks:**\n' +
      '\t+ Create a personalized study schedule\n' +
      '\t+ Identify specific areas of data structures to focus on\n' +
      '\t+ Adjust study schedule as needed\n' +
      '\n' +
      '**Key Takeaways:**\n' +
      '\n' +
      '* The user values quality over quantity in their study sessions.\n' +
      '* They prioritize regular breaks to maintain productivity and avoid burnout.\n' +
      '* The user is looking for a personalized study schedule and guidance on focusing on specific areas of data structures.'
  }
]
data236_hw6>
```

6. Episodes

```
]
data236_hw6> db.episodes.find(
|   { user_id: "Kruthika" },
|   { embedding: 0 }
| ).sort({ created_at: -1 }).pretty()
[
  {
    _id: ObjectId('69b99aefad4b7f7c7a5de3bc'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: 'Pack essential items like chargers, adapters, and medication in your carry-on luggage.',
    importance: 0.7,
    created_at: ISODate('2026-03-17T18:18:23.922Z')
  },
  {
    _id: ObjectId('69b99aefad4b7f7c7a5de3bb'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: 'Make digital copies of important documents and leave them with a trusted friend or family member.',
    importance: 0.9,
    created_at: ISODate('2026-03-17T18:18:23.896Z')
  },
  {
    _id: ObjectId('69b99aefad4b7f7c7a5de3ba'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: 'Double-check the weather forecast for your destination and pack accordingly.',
    importance: 0.8,
    created_at: ISODate('2026-03-17T18:18:23.868Z')
  },
  {
    _id: ObjectId('69b99a99ad4b7f7c7a5de3b7'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: "Some students may need more or less study time, so it's essential to find a balance that works for you and your schedule.",
    importance: 0.9,
    created_at: ISODate('2026-03-17T18:16:57.514Z')
  },
  {
    _id: ObjectId('69b99a99ad4b7f7c7a5de3b6'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: 'The American Psychological Association recommends 30 minutes of focused study time followed by a 10-15 minute break.',
    importance: 0.7,
    created_at: ISODate('2026-03-17T18:16:57.487Z')
  },
  {
    _id: ObjectId('69b99a99ad4b7f7c7a5de3b5'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: 'The amount of study time varies by individual and subject, but 1-2 hours per day is a common range for most students.',
    importance: 0.8,
    created_at: ISODate('2026-03-17T18:16:57.457Z')
  },
  {
    _id: ObjectId('69b99a62ad4b7f7c7a5de3b2'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: 'Stacks and Queues',
    importance: 0.6,
    created_at: ISODate('2026-03-17T18:16:02.213Z')
  },
  {
    _id: ObjectId('69b99a62ad4b7f7c7a5de3b1'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: 'Linked Lists',
    importance: 0.7,
    created_at: ISODate('2026-03-17T18:16:02.117Z')
  }
]
```

```

    importance: 0.6,
    created_at: ISODate('2026-03-17T18:16:02.213Z')
  },
  {
    _id: ObjectId('69b99a62ad4b7f7c7a5de3b1'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: 'Linked Lists',
    importance: 0.7,
    created_at: ISODate('2026-03-17T18:16:02.117Z')
  },
  {
    _id: ObjectId('69b99a62ad4b7f7c7a5de3b0'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: 'Arrays',
    importance: 0.8,
    created_at: ISODate('2026-03-17T18:16:02.090Z')
  },
  {
    _id: ObjectId('69b999b0ad4b7f7c7a5de3ad'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: 'Prioritize the most challenging subjects or topics before less demanding ones.',
    importance: 0.7,
    created_at: ISODate('2026-03-17T18:13:04.241Z')
  },
  {
    _id: ObjectId('69b999b0ad4b7f7c7a5de3ac'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: 'Create a schedule with dedicated blocks for reviewing notes and practicing problems.',
    importance: 0.9,
    created_at: ISODate('2026-03-17T18:13:04.213Z')
  },
  {
    _id: ObjectId('69b999b0ad4b7f7c7a5de3ab'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: 'Set specific and achievable goals for each study session.',
    importance: 0.8,
    created_at: ISODate('2026-03-17T18:13:04.187Z')
  },
  {
    _id: ObjectId('69b998a8ac774851a74fc4f1'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: 'Prioritize the most challenging subjects or topics before less demanding ones.',
    importance: 0.7,
    created_at: ISODate('2026-03-17T18:08:40.012Z')
  },
  {
    _id: ObjectId('69b998a7ac774851a74fc4f0'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: 'Create a schedule with dedicated blocks for reviewing notes and practicing problems.',
    importance: 0.9,
    created_at: ISODate('2026-03-17T18:08:39.984Z')
  },
  {
    _id: ObjectId('69b998a7ac774851a74fc4ef'),
    user_id: 'Kruthika',
    session_id: 'Planning phase',
    fact: 'Set specific and achievable goals for each study session.',
    importance: 0.8,
    created_at: ISODate('2026-03-17T18:08:39.953Z')
  }
]
data236 hw6> 

```

```
data236_hw6> db.messages.countDocuments({})
25
data236_hw6> db.summaries.countDocuments({})
2
data236_hw6> db.episodes.countDocuments({})
26
data236_hw6> █
```

The endpoints (/api/chat, /api/memory, /api/aggregate).

```
from fastapi import APIRouter, HTTPException
from pymongo.errors import PyMongoError

from ..config import settings
from ..schemas import AggregateResponse, ChatRequest, ChatResponse, MemoryViewResponse
from ..services.memory_service import (
    aggregate_user_activity,
    build_chat_messages,
    get_latest_summary,
    get_recent_messages,
    load_memory_view,
    maybe_refresh_summaries,
    normalize_session_id,
    retrieve_relevant_episodes,
    save_message,
    store_episodes_for_message,
)
from ..services.ollama_service import OllamaError, chat as ollama_chat

router = APIRouter(prefix="/api", tags=["Memory"])

@router.post("/chat", response_model=ChatResponse)
def chat(payload: ChatRequest):
    session_id = normalize_session_id(payload.session_id)

    try:
        save_message(payload.user_id, session_id, "user", payload.message)

        short_term = get_recent_messages(
            payload.user_id,
            session_id,
            limit=settings.SHORT_TERM_N + 1,
```

```

    )

    context_window = short_term
    if (
        context_window
        and context_window[-1]["role"] == "user"
        and context_window[-1]["content"] == payload.message
    ):
        context_window = context_window[:-1]

    session_summary_doc = get_latest_summary(payload.user_id, "session",
session_id=session_id)
    lifetime_summary_doc = get_latest_summary(payload.user_id, "user")

    episodic = retrieve_relevant_episodes(
        payload.user_id,
        payload.message,
        top_k=settings.EPISODIC_TOP_K,
    )

    model_messages = build_chat_messages(
        user_message=payload.message,
        short_term_messages=context_window,
        lifetime_summary=(lifetime_summary_doc or {}).get("text"),
        session_summary=(session_summary_doc or {}).get("text"),
        episodic_facts=episodic,
    )

    reply = ollama_chat(settings.CHAT_MODEL, model_messages)
    save_message(payload.user_id, session_id, "assistant", reply)

    extracted_episodes = store_episodes_for_message(payload.user_id, session_id,
payload.message)
    summary_updates = maybe_refresh_summaries(payload.user_id, session_id)

    return {
        "reply": reply,
        "memory_used": {
            "short_term_count": len(short_term),
            "short_term_messages": short_term,
            "long_term_used": {
                "session_summary": (session_summary_doc or {}).get("text"),

```

```

        "lifetime_summary": (lifetime_summary_doc or {}).get("text"),
    },
    "retrieved_episodic_facts": episodic,
    "newly_extracted_episodes": extracted_episodes,
    "summary_updates": summary_updates,
},
}
except OllamaError as exc:
    raise HTTPException(status_code=502, detail=str(exc))
except PyMongoError:
    raise HTTPException(status_code=500, detail="Database error during chat
processing.")

@router.get("/memory/{user_id}", response_model=MemoryViewResponse)
def memory_view(user_id: str, session_id: str | None = None):
    try:
        sid = normalize_session_id(session_id)
        view = load_memory_view(user_id, sid)
        return view
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while fetching
memory.")

@router.get("/aggregate/{user_id}", response_model=AggregateResponse)
def aggregate_view(user_id: str):
    try:
        return aggregate_user_activity(user_id)
    except PyMongoError:
        raise HTTPException(status_code=500, detail="Database error while aggregating
memory.")

```

The memory logic functions: (episode extraction, summarization trigger, embedding + retrieval, prompt assembly).

```

import json
import re

from ..config import settings

```

```

from ..db import (
    get_episodes_collection,
    get_messages_collection,
    get_summaries_collection,
)
from ..utils import cosine_similarity, now_utc
from .ollama_service import OllamaError, chat as ollama_chat, embed as ollama_embed

def normalize_session_id(session_id: str | None) -> str:
    if session_id and session_id.strip():
        return session_id.strip()
    return settings.DEFAULT_SESSION_ID

def save_message(user_id: str, session_id: str, role: str, content: str) -> dict:
    doc = {
        "user_id": user_id,
        "session_id": session_id,
        "role": role,
        "content": content,
        "created_at": now_utc(),
    }
    get_messages_collection().insert_one(doc)
    return doc

def get_recent_messages(user_id: str, session_id: str, limit: int) -> list[dict]:
    docs = list(
        get_messages_collection()
        .find({"user_id": user_id, "session_id": session_id})
        .sort("created_at", -1)
        .limit(limit)
    )
    docs.reverse()

    return [
        {
            "role": d.get("role", "user"),
            "content": d.get("content", ""),
            "created_at": d.get("created_at"),
        }
    ]

```



```

        for d in docs
    ]

def get_latest_summary(user_id: str, scope: str, session_id: str | None = None) -> dict | None:
    query = {"user_id": user_id, "scope": scope}
    if scope == "session":
        query["session_id"] = session_id
    else:
        query["session_id"] = None

    return get_summaries_collection().find_one(query, sort=[("created_at", -1)])

def _strip_code_fences(text: str) -> str:
    cleaned = text.strip()
    if cleaned.startswith("```"):
        cleaned = re.sub(r"^```[a-zA-Z0-9]*", "", cleaned)
        cleaned = cleaned.rstrip("`").strip()
    return cleaned

def _try_parse_json(raw_text: str):
    cleaned = _strip_code_fences(raw_text)

    try:
        return json.loads(cleaned)
    except json.JSONDecodeError:
        pass

    match = re.search(r"\[([\s\S]*\]", cleaned)
    if match:
        try:
            return json.loads(match.group(0))
        except json.JSONDecodeError:
            return None

    match = re.search(r"\{([\s\S]*\}", cleaned)
    if match:
        try:
            return json.loads(match.group(0))

```

```

        except json.JSONDecodeError:
            return None

    return None

def _fallback_fact_extraction(message: str) -> list[dict]:
    parts = [p.strip() for p in re.split(r"[!?]\s+", message) if p.strip()]
    facts = []

    for part in parts:
        if len(part) < 8:
            continue
        facts.append({"fact": part[:180], "importance": 0.5})
        if len(facts) == 3:
            break

    if not facts and message.strip():
        facts.append({"fact": message.strip()[:180], "importance": 0.5})

    return facts

def extract_episode_candidates(message: str) -> list[dict]:
    system_prompt = (
        "Extract up to 3 useful user facts for future conversation memory. "
        "Return JSON array only. Each item must have keys: fact (string) and importance "
        "(float 0..1). "
    )

    user_prompt = (
        f"User message: {message}\n\n"
        "Rules:\n"
        "- Keep facts short and concrete.\n"
        "- Skip generic chatter.\n"
        "- importance should be 0.0 to 1.0.\n"
        "- Return only JSON array."
    )

    raw = ollama_chat(
        settings.CHAT_MODEL,
        [

```

```

        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ],
    temperature=0.0,
)

parsed = _try_parse_json(raw)
if not isinstance(parsed, list):
    return _fallback_fact_extraction(message)

facts = []
for item in parsed:
    if not isinstance(item, dict):
        continue

    fact = str(item.get("fact", "")).strip()
    if not fact:
        continue

    try:
        importance = float(item.get("importance", 0.5))
    except (TypeError, ValueError):
        importance = 0.5

    importance = max(0.0, min(1.0, importance))
    facts.append({"fact": fact[:220], "importance": importance})

    if len(facts) == 3:
        break

return facts or _fallback_fact_extraction(message)

def store_episodes_for_message(user_id: str, session_id: str, message: str) ->
list[dict]:
    episodes_col = get_episodes_collection()
    facts = extract_episode_candidates(message)
    inserted = []

    for fact_obj in facts:
        emb = ollama_embed(fact_obj["fact"])
        doc = {
            "user_id": user_id,

```

```

        "session_id": session_id,
        "fact": fact_obj["fact"],
        "importance": fact_obj["importance"],
        "embedding": emb,
        "created_at": now_utc(),
    }
    episodes_col.insert_one(doc)
    inserted.append({
        "fact": doc["fact"],
        "importance": doc["importance"],
    })

return inserted

def retrieve_relevant_episodes(user_id: str, message: str, top_k: int) -> list[dict]:
    query_vec = ollama_embed(message)
    candidates = list(
        get_episodes_collection()
        .find({"user_id": user_id})
        .sort("created_at", -1)
        .limit(250)
    )

    scored = []
    for item in candidates:
        emb = item.get("embedding")
        if not isinstance(emb, list):
            continue

        try:
            emb = [float(v) for v in emb]
        except (TypeError, ValueError):
            continue

        sim = cosine_similarity(query_vec, emb)
        if sim <= 0.0:
            continue

        scored.append(
            {
                "fact": item.get("fact", ""),

```

```

        "importance": float(item.get("importance", 0.0)),
        "similarity": round(sim, 4),
        "created_at": item.get("created_at"),
    }
)

scored.sort(key=lambda x: x["similarity"], reverse=True)
return scored[:top_k]

def build_chat_messages(
    user_message: str,
    short_term_messages: list[dict],
    lifetime_summary: str | None,
    session_summary: str | None,
    episodic_facts: list[dict],
) -> list[dict]:
    episodic_line = "; ".join(
        f"{e['fact']} (imp={e['importance']:.2f}, sim={e['similarity']:.2f})"
        for e in episodic_facts
    )
    if not episodic_line:
        episodic_line = "None"

    system_context = (
        "You are a concise and helpful assistant. Use memory context when useful, "
        "but do not hallucinate details that are not present.\n\n"
        f"Lifetime summary:\n{lifetime_summary or 'None'}\n\n"
        f"Session summary:\n{session_summary or 'None'}\n\n"
        f"Top episodic facts:\n{episodic_line}"
    )

    messages = [{"role": "system", "content": system_context}]
    messages.extend(
        {"role": m["role"], "content": m["content"]}
        for m in short_term_messages
    )
    messages.append({"role": "user", "content": user_message})
    return messages

def _conversation_to_text(messages: list[dict]) -> str:

```

```

        return "\n".join(f"{m['role']}: {m['content']}" for m in messages)

def maybe_refresh_summaries(user_id: str, session_id: str) -> dict | None:
    msg_col = get_messages_collection()
    user_count = msg_col.count_documents(
        {"user_id": user_id, "session_id": session_id, "role": "user"}
    )

    every_n = max(1, settings.SUMMARIZE_EVERY_USER_MSGS)
    if user_count == 0 or user_count % every_n != 0:
        return None

    recent_messages = get_recent_messages(user_id, session_id, limit=28)
    if not recent_messages:
        return None

    summary_prompt = (
        "Summarize this conversation into concise bullets. Include user goals, "
        "preferences, and unresolved tasks. Keep under 8 bullets."
    )

    convo_text = _conversation_to_text(recent_messages)

    session_summary_text = ollama_chat(
        settings.CHAT_MODEL,
        [
            {"role": "system", "content": summary_prompt},
            {"role": "user", "content": convo_text},
        ],
        temperature=0.1,
    )

    summaries_col = get_summaries_collection()
    summaries_col.update_one(
        {"user_id": user_id, "session_id": session_id, "scope": "session"},
        {
            "$set": {
                "text": session_summary_text,
                "created_at": now_utc(),
            }
        },
        upsert=True,
    )

```

```

    )

    lifetime_summary_text = refresh_lifetime_summary(user_id)

    return {
        "session_summary": session_summary_text,
        "lifetime_summary": lifetime_summary_text,
    }

def refresh_lifetime_summary(user_id: str) -> str:
    summaries_col = get_summaries_collection()
    session_summaries = list(
        summaries_col.find({"user_id": user_id, "scope": "session"})
        .sort("created_at", -1)
        .limit(8)
    )

    if not session_summaries:
        return ""

    joined = "\n\n".join(
        f"Session summary {idx + 1}: {s.get('text', '')}"
        for idx, s in enumerate(session_summaries)
    )

    lifetime_prompt = (
        "Create a compact long-term user profile summary from session summaries. "
        "Focus on durable preferences, goals, and recurring patterns. Keep it concise."
    )

    lifetime_summary_text = ollama_chat(
        settings.CHAT_MODEL,
        [
            {"role": "system", "content": lifetime_prompt},
            {"role": "user", "content": joined},
        ],
        temperature=0.1,
    )

    summaries_col.update_one(
        {"user_id": user_id, "scope": "user", "session_id": None},

```

```

        {
            "$set": {
                "text": lifetime_summary_text,
                "created_at": now_utc(),
            }
        },
        upsert=True,
    )

    return lifetime_summary_text


def load_memory_view(user_id: str, session_id: str) -> dict:
    session_summary = get_latest_summary(user_id, "session", session_id=session_id)
    lifetime_summary = get_latest_summary(user_id, "user")

    episodes = list(
        get_episodes_collection()
        .find({"user_id": user_id, "session_id": session_id})
        .sort("created_at", -1)
        .limit(20)
    )

    return {
        "user_id": user_id,
        "session_id": session_id,
        "recent_messages": get_recent_messages(user_id, session_id, limit=16),
        "latest_session_summary": session_summary,
        "latest_lifetime_summary": lifetime_summary,
        "recent_episodes": [
            {
                "fact": e.get("fact", ""),
                "importance": float(e.get("importance", 0.0)),
                "created_at": e.get("created_at"),
            }
            for e in episodes
        ],
    }


def aggregate_user_activity(user_id: str) -> dict:
    msg_col = get_messages_collection()

```



```

summaries_col = get_summaries_collection()

pipeline = [
    {"$match": {"user_id": user_id}},
    {
        "$group": {
            "_id": {
                "$dateToString": {
                    "format": "%Y-%m-%d",
                    "date": "$created_at",
                }
            },
            "count": {"$sum": 1},
        }
    },
    {"$sort": {"_id": 1}},
]

daily = [
    {"date": row["_id"], "count": row["count"]}
    for row in msg_col.aggregate(pipeline)
]

summaries = list(
    summaries_col.find({"user_id": user_id}).sort("created_at", -1).limit(6)
)

return {
    "user_id": user_id,
    "daily_message_counts": daily,
    "recent_summaries": [
        {
            "user_id": s.get("user_id"),
            "session_id": s.get("session_id"),
            "scope": s.get("scope"),
            "text": s.get("text"),
            "created_at": s.get("created_at"),
        }
        for s in summaries
    ],
}

```

